

## Patata

- **Límite de tiempo:** 10 minutos
- **Entorno:** una GPU (≈16 GB VRAM), sin internet
- **Tamaño de la solución:** `solution.ipynb` ≤ 1 MB
- **Almacenamiento:** 5 GB

## Tarea

Tu amigo propone jugar a un juego de adivinanzas. Él, como juez, elige una palabra oculta de un vocabulario fijo, y tu debes encontrarla en 30 turnos como máximo. En cada turno, el juez compara dos palabras e indica cuál está semánticamente más cerca de la palabra oculta. Cada partida comienza con el par fijo `lamp` vs `potato` (lampara y patata), porque son dos de las cosas favoritas de tu amigo. A continuación, tu programa propone una palabra nueva. La palabra ganadora de la comparación se conserva y se compara con su siguiente propuesta. Ganas una partida en el momento en que propones exactamente la palabra oculta. La comparación no distingue entre mayúsculas y minúsculas. Cada palabra que propongas debe estar en `dataset/vocabulary.json`.

Hay un ejemplo completo en `solution.ipynb` con el protocolo y la carga de datos. Puedes modificar la clase `PublicEmbeddingPlayer`. Tu programa se inicializa una vez y juega todas las partidas en una sola ejecución; el protocolo crea una instancia nueva de `PublicEmbeddingPlayer` al comienzo de cada partida.

## El juez

Tu programa envía un objeto JSON al juez y el juez responde con un objeto JSON.

Un ejemplo desarrollado, en el que la palabra oculta se muestra únicamente para explicar el protocolo:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

Los turnos se indexan del 1 al 30.

Las opciones de `verdict` son `first`, que significa que `word1` está más cerca; `second`, que significa que `word2` está más cerca; o `same`, que significa que ambas palabras están igual de cerca de la palabra oculta.

`winner_word` es la palabra que se conserva para la siguiente comparación. Con un veredicto `same`, la primera palabra permanece.

## Dataset

Compartido por todas las particiones:

- `dataset/vocabulary.json` — 1602 palabras únicas en minúsculas. La palabra oculta siempre es una de ellas.
- `dataset/public_embeddings.npy` — `float32`, con forma (1602, 2560). La fila `i` corresponde a la palabra `i` del vocabulario. Estos son *embeddings públicos*; el juez utiliza una representación privada diferente.

Las particiones son conjuntos de palabras ocultas:

| Partición                       | Palabras | Respuestas                            | Úsela para                           |
|---------------------------------|----------|---------------------------------------|--------------------------------------|
| <code>test_public</code>        | 120      | <code>dataset/test_public.json</code> | ejecutar su solución y autoevaluarla |
| <code>test_leaderboard_a</code> | 120      | ocultas                               | clasificación en vivo                |
| <code>test_leaderboard_b</code> | 120      | ocultas                               | clasificación final                  |

No hay ninguna partición `train`; no se ajusta nada a partir de filas etiquetadas.

## Modelos proporcionados

Con la tarea se incluyen dos modelos de embeddings preentrenados que pueden utilizarse:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Ambos deben cargarse desde su ruta local; un identificador del hub de Hugging Face como "BAAI/bge-m3" activa una descarga y falla, porque la evaluación se realiza sin conexión. Cada directorio contiene un `example.py` ejecutable que muestra la llamada sin conexión.

Bibliotecas disponibles: `numpy`, `torch`, `sentence-transformers`. Sin internet, sin descargas y sin otros paquetes.

## Salida

Ninguna. Esta es una tarea interactiva: tu solución no escribe ningún archivo de respuesta; se comunica con el juez mediante `stdin/stdout` como se ha descrito anteriormente.

## Métrica

Una partida resuelta en el turno  $t$  obtiene  $1.0 - 0.02 \times \max(0, t - 10)$ ; una partida no resuelta en 30 turnos obtiene 0. Por tanto, los turnos 1-10 obtienen 1.00, el turno 20 obtiene 0.80 y el turno 30 obtiene 0.60.

La puntuación de la tarea es la puntuación media de las partidas  $\times 100$ , entre 0.00 y 100.00.

El límite de 10 minutos es un único presupuesto que abarca el inicio, la preparación y las 120 partidas del conjunto de prueba.

## Cómo enviar

1. Abre `solution.ipynb`, edita `PublicEmbeddingPlayer` y ejecuta todas las celdas para asegurarte de que funciona.
2. Opcionalmente, compruébalo localmente: `python local_test.py solution.ipynb --limit 5`. El juez local utiliza los embeddings *públicos*, por lo que su puntuación es solo orientativa.
3. Guarda `solution.ipynb`.
4. Abre la pestaña Git en la barra lateral izquierda de JupyterLab.
5. Añade `solution.ipynb` al área de preparación (el icono + situado junto a él).
6. Introduce un mensaje de commit y haz click en Commit.
7. Haz click en la nube con una flecha hacia arriba para hacer push.
8. Vuelve a esta página del concurso y haz click en Submit, con un mensaje de commit que coincida con el que has proporcionado.

Envía exactamente un archivo, llamado `solution.ipynb`, que abarque cualquier preparación e inferencia necesarias.